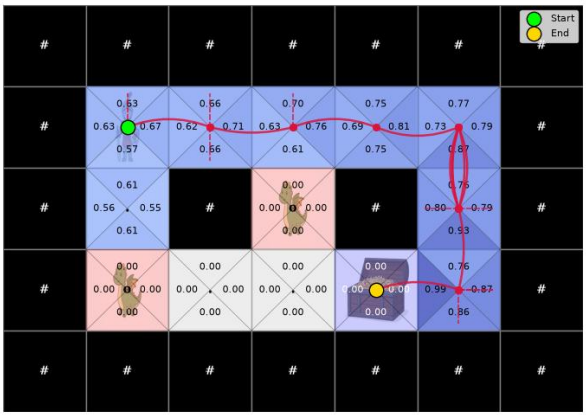
IML Assignment 2

Responsible: Arthicha Srisuchinnawong (P1)

Materials: GitHub
Due: xxxxx (end of day)

Part 1: An RL knight in a gridworld (10 pts)



Preamble

Congratulations! You have been knighted. Your first mission as a knight is to navigate various dungeons and bring back treasure to help our kingdom prosper. The aim of this first mission (and this problem) is to help you develop an intuition for the different parameters of a simple reinforcement learning agent, particularly a Q-learning agent. You will also experience a problem called reward hacking through trial and error, which is a (hopefully) fun challenge in reinforcement learning.

The `Dungeon` class in `dungeon.py` defines a simple gridworld environment, created by using maps generated in `dungeon_maker.py`. The `Agent` class in `agent.py` defines the Q-learning agent (the knight) to be programmed externally (see coding template), and the `DungeonVisualization` class in `visualization.py` provides the necessary plots to observe and interpret the agent's strategies.

States represent positions within a dungeon. Some states are unreachable because they are occupied by walls (#) or dragons (o). When you try to walk into a wall, your position remains unchanged. When you try to walk into a dragon, you are eliminated and respawned at the starting position. You can take four possible integer actions $\{0, 1, 2, 3\}$, each representing a direction in which the agent chooses to move. Your goal is to navigate from the starting position `start` (s) to the goal position `end` (e, representing the treasure) as 'optimally' as possible.

Remember that a policy is a rule specifying which actions the agent should take as a (potentially stochastic) function of its current state. For a given policy $\pi(a|s)$, or simply π, let $V_\pi(i)$ be the expected return that an agent following the policy will receive per episode (running for a specific number of steps).

1.1 A drunken knight in a treasure room (4 pts)

use default `dungeon = make_treasure_room()` without slippery.

(a) Implement a stochastic policy π where the you select actions uniformly at random. (0.5 pts)

Hint: You may use `agent.run_episode(dungeon, nstep, gamma = 1)` to run the agent and retrieve the state, action, and reward trajectory.

[provide your code here]

(b) Observe how you move and draw a Markov decision process (MDP) diagram for all four non-wall states.
Hint: Refer to Lecture 8, Slide 35 (40/48). (0.5 pts)

[draw, capture, and insert your photo/figure here]

(c) Compute the theoretical probability distribution over a 2-step horizon analytically (without writing code) when you use the stochastic policy you implemented. (1 pt)
Hint: In conditional probability and Markov processes, the probability distribution after N steps is obtained by marginalizing over all intermediate state trajectories:

$$P(s^{(N)} = s_N \mid s^{(0)} = s_0) = \sum_{s_1,\ldots,s_{N-1}} \prod_{t=1}^{N} P(s^{(t)} = s_t \mid s^{(t-1)} = s_{t-1})$$

where $s^{(t)}$ denotes the state at time step t. You can compute the N-step distribution by taking the product of single-step transition probabilities and summing over all possible intermediate states $s_1, \ldots, s_{N-1}$.

(d) If you implement Q-learning with an episode length of 1 (`nstep = 1`) under current setup, will you manage to perform better? Why or why not? Explain your reasoning. (1 pt)
Hint: You may use your answer to the previous question (and are encouraged to implement it to verify your understanding).

(e) For a dungeon of width N and height M, what is the minimum number of steps required to reach the goal from the starting position? Why? (1 pt)
Hint: You may use your answer to the previous question (and are encouraged to implement it to verify your understanding).

1.2 A tricky knight in an underground maze (6 pts)

use default `dungeon = make_underground_maze()` without slippery.

(a) Using your stochastic policy π, implement the Q-learning update with a discount factor ($\gamma = 1$) and a maximum episode length of 30 steps ($n_{\text{step}} = 30$). Run an initial episode, followed by several consecutive episodes without resetting the agent's Q-table. Explain your observations on how the Q-values change relative to the goal and starting locations. (1 pt)

Hint: Consider the evolution of the Q-table, success rate, and state coverage. You may also use `<vis>.show(states, actions, <knight>.q_values)` to visualize your walking strategy and `<vis>.plot_training_reward()` to monitor your performance.

[Insert ≈ 3 selected snapshots (without resetting the agent) as your empirical proof here]

(b) By varying the discount factor γ (e.g., trying values such as 1.0, 0.99, 0.9, $\cdots$, 0.0), do you observe any differences? Summarize how the discount rate affects the results and explain why it is necessary. (1 pt)

Hint: What happens to the agent's behavior at low, high, and balanced γ values?

[Insert at least 2 selected snapshots as your empirical proof here]

(c) Implement an ϵ-greedy policy for your agent by varying ϵ (e.g., trying values such as 0.0, 0.2, 0.5, 0.8, 0.99, and 1.0). Plot the sum of discounted rewards per episode over 100 training runs and explain your observations. (1 pt)

Hint: You may use `<vis>.show(states, actions, <knight>.q_values)` to visualize your walking strategy and `<vis>.plot_training_reward()` to monitor your performance.

[Insert your code and at least 3 selected snapshots as empirical proof here]

(d) Will you reach the goal every time? Explain why or why not. What should we do when this happens. (0.5 pts)

(e) Recreate the maze using `make_underground_maze_with_meat()`, where you get half of the reward when stepping into the meat states (+); re-implemented the reward function; set $\epsilon = 0.9$, $n_{\text{step}} = 50$, and $\gamma = 0.95$, and run the agent for 100 to 200 training runs. Explain your observations regarding your strategy (Q-values) and behavior. (1 pt)

(f) Compare the state values $V(s)$ when you go directly to the treasure (e), collects free meat as you observed (+), or do both. Explain your results mathematically. (1.5 pts)
Hint: Consider the Bellman Optimality Equation recursively:

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right)$$

where $R(s,a)$ is the reward received for taking action a in state s, γ is the discount factor, and s' is the subsequent state resulting from the transition. You may assume a deterministic policy and state transition (the best-case scenario).

(g) How might you modify the reward function or environment design to prevent this *reward hacking*? Propose and explain a solution. (Extra, +0.5 pts)
Hint: you should refer to your previous answer(s).